MMA v2 — полный список возможностей с подробностями

> Обзор возможностей агента простыми словами. Технические детали — в AGENTS.md и docs/specs/.

---

1. Архитектура: 6-слойная машина состояний

Агент устроен как конвейер из слоёв, где каждый занимается своим делом:

```
CLI → Core → LLM-слой → Инструменты → Модули
```

Как проходит один ход агента (цикл состояний внутри Core):

1. **INIT** — при старте собирается системный промпт из блоков (инструкции, карта проекта, память, план), строится история диалога.
2. **THINK** — запрос к модели. Ответ проходит проверку на «галлюцинации» до того, как будет показан.
3. **ACT** — если модель решила вызвать инструмент (прочитать файл, запустить команду) — вызов парсится и выполняется через исполнитель.
4. **OBSERVE** — результат инструмента возвращается в контекст; система решает: продолжать цикл или завершаться.
5. **OUTPUT** — модель выдала финальный текст без вызовов инструментов → текст показывается пользователю.
6. **ERROR** — при ошибке: повтор или эскалация, а не падение всей программы.

Ключевое: это не «большой запутанный объект», а простая машина состояний — легко понять, на каком этапе что пошло не так. Каждый слой можно тестировать отдельно.

---

2. Инструменты (34)

Группа	Инструменты	Что делают
Файлы	`read_file`, `write_file`, `edit_file`, `list_dir`, `glob`, `grep`, `file_info`, `move_file`, `delete_file`, `create_dir`	Чтение (по 300 строк за раз, чтобы не жечь контекст), запись с автосозданием папок, поиск по имени (`glob`) и по содержимому (`grep`)
Команды	`bash`	Запуск команд терминала. Долгие процессы (сервер разработки и т.п.) через ~5 секунд автоматически уходят в фон и получают ID
Фоновые процессы	`process_list`, `process_log`, `process_kill`	Список фоновых процессов, чтение их вывода, остановка дерева процессов
Интернет	`web_search`, `web_fetch`, `web_browse`	Поиск, чтение страниц (без JS), полноценный браузер
Загрузки	`download_file`	Скачать бинарный файл (картинку, архив) сразу на диск — байты не попадают в контекст
Браузер	`browser`	Playwright: открыть сайт, кликнуть по элементу, ввести текст, скролл, скриншот. Элементы нумеруются, агент работает по номерам (`click(3)`)
Мультимодальность	`attach_image`	Прикрепить картинку (файл/URL/буфер обмена); большие PNG пережимаются в маленький JPEG, чтобы влезть в контекст
Делегирование	`subagent`	Порождение субагента для подзадачи (с зоной доступа к файлам)
Обработка текста	`chunk_query`	Разрезать большой текст на куски и обработать параллельно
Расширения	`mcp_call`, `pipeline_run`, `load_skill`	Вызов внешнего инструмента MCP-сервера, запуск многошагового сценария (pipeline), загрузка скилла по имени
Память	`remember`, `recall`	«Запомни…» / «что помнишь о…» — явная запись и чтение постоянной памяти (регистрируются модулем памяти)
История	`search_history`	Полнотекстовый поиск по всем прошлым сессиям
Служебный	`enable_tools`	Включение скрытых инструментов на лету (см. раздел 3)
Структурные (от модулей)	`plan`, `todo`, `verify`, `project_map`, `lsp_check`	Управление планом/подзадачами, проверка исполнения, запрос карты

Группа	Инструменты	Что делают
		проекта, проверка ошибок компиляции через LSP

Примечание: инструменты `question` / `approve` (интерактивные меню-вопросы) существуют в коде, но отключены — они сбивали маленькие модели; вместо них модель задаёт вопросы обычным текстом.

---

3. Сужение набора инструментов

Проблема: каждая схема инструмента тратит токены контекста. 28 инструментов ≈ 3500 токенов на каждый запрос модели.

Решение:

- По умолчанию агент видит только **17 нужных чаще всего** (~2100 токенов, −40%).
- Остальные («скрытые»: веб, браузер, картинки, процессы) перечислены в отдельном блоке промпта — модель знает, что они есть.
- Когда они понадобятся, модель вызывает `enable_tools ["research"]` — инструменты появятся со следующего хода.
- Структурные инструменты (план, проверка) всегда видны — их нельзя случайно отключить.

---

4. Контекстный бюджет и компакция

У маленькой модели мало «рабочей памяти» — она расходуется бережно:

- **Динамические бюджеты** от размера окна модели (32K → 3K инструкции / 5K запас / 24K история; 64K → 6/10/48 и т.д.).
- **Компакция** срабатывает при заполнении 75% бюджета истории или каждые 15 ходов: старые сообщения сворачиваются в **структурное резюме**, которое сохраняет самое важное — исходную задачу пользователя, какие файлы созданы/изменены/удалены, состояние плана, ключевые решения, ошибки и то, что уже пробовали безуспешно.
- Последние 6 обменов остаются полностью — свежий контекст не теряется.
- Резюме пишется в лог один раз (без дублирования).
- Подсчёт токенов точный (tiktoken), с грубым запасным вариантом.
- **Наблюдаемость**: в лог сессии пишутся события `context` (структура и заполненность перед каждым запросом) и `compaction` (что, когда и почему сжалось) — можно восстановить всю историю деградации контекста постфактум.

Подробнее о том, почему после десятков сжатий агент всё ещё помнит задачу — см. раздел «Как агент не теряет суть проекта».

---

5. Планирование и гарантии исполнения

- **План-чеклист живёт прямо в системном промпте**: `[x] сделано / [ ] ждёт`. Модель видит его каждый ход и не может «забыть» задачу.
- Решение создавать план принимает сама модель (по сложности задачи), а не хрупкие эвристики по ключевым словам.
- **Хранилище планов**: активный план + черновики + архив. Создание нового не уничтожает старый — незавершённый уходит в черновики. Есть `re-plan` (сохранить сделанное, перепланировать остаток), удаление, очистка.
- **Самосертификация шагов запрещена**: пометить шаг «готово» можно только если названные в нём файлы реально существуют (для шагов-удалений — что файлов больше нет). Шаг с ошибкой компиляции «закрыть» тоже нельзя.
- **Детектор застревания**: слишком много итераций на одном шаге, три одинаковых вызова подряд, серия неудачных bash-команд, долгие серии только-чтения без записи — всё это вызывает подсказку «смени подход».
- **Финальный аудит-шлюз**: когда модель пытается выдать финальный ответ при активном плане, система проверяет — все ли шаги закрыты, все ли файлы на месте, проходит ли проверка типов и тесты. Если нет — в контекст летит предупреждение, и работа продолжается (ограничено лимитом итераций).
- После каждой записи файла автоматически гоняется быстрая проверка синтаксиса/типов, ошибки показываются модели немедленно.

---

6. Надёжность связи с LLM

Малые модели через локальные бэкенды (llama.cpp, LM Studio) капризны — связь укреплена со всех сторон:

- **Повторы с экспоненциальной паузой** при 429 (перегрузка) и ошибках сервера; уважается заголовок `Retry-After`.
- **Стриминг с fallback**: если стрим не вернул ни текста, ни вызова инструмента — автоматический повтор тем же запросом в обычном (не потоковом) режиме, который надёжнее для tool-calls.
- **Обрыв потока до первого байта** — тихо повторяется; после начала выдачи контента — нет (иначе задублируются куски).
- **Тайм-аут тишины**: молчаливый сервер не съедает весь тайм-аут запроса.
- **Обрезанные ответы** распознаются (`finish_reason: length`) и не маскируются под «пустой ответ» — модель получает понятную ошибку и адаптируется (например, режет большую запись на части).
- **Переключение провайдеров (failover)**: основной провайдер лёг (429/сеть) → автоматический переход на следующий из настроенных; удавшийся становится основным. Есть проверка доступности всех провайдеров (`mma provider check`).
- **Hot-swap**: сменить модель/провайдера посреди диалога командой, без перезапуска.

7. MoE — разложение задачи по субагентам

«Mixture of Experts» на уровне агента: одна большая задача → несколько специалистов. Вместо того чтобы одна маленькая модель тащила всё сразу (и путалась), задача делится, и каждую часть делает отдельный субагент со своим набором полномочий.

Как это работает, шаг за шагом

1. **Router строит план.** Отдельная модель-оркестратор (обычно покрупнее рабочей) превращает запрос пользователя в структурированный план из подзадач: что сделать, какие файлы затронуть, от чего что зависит.
2. **План проверяется на здравость** до исполнения:
   - *циклы* («А зависит от Б, а Б от А») — детектируются;
   - *конфликт записей* — две подзадачи одновременно пишут в один файл;
   - *нарушенные зависимости* — подзадача читает файл, который появится только позже (read-after-write);
   - отсутствующие зависимости достраиваются автоматически; неисправимое → план на переделку.
3. **Каждому эксперту — свои полномочия.** Эксперты описываются в конфиге: своя модель, свой набор инструментов (по тегам — например, «файловому» эксперту не дают веба) и лимит попыток. Субагенту выдаётся **зона доступа к файлам**: назначенные файлы можно менять, остальные — в лучшем случае читать.
4. **Параллельные волны.** Подзадачи сортируются топологически по зависимостям и исполняются волнами: сначала независимые (параллельно), потом те, кто их дождался, и так до конца.
5. **Устойчивость к сбоям.** Временные ошибки (тайм-аут, перегрузка сервера, сеть) ретраятся с паузой до лимита попыток эксперта. Структурный провал (субагент сделал не то) не убивает процесс — он уходит оркестратору как сигнал для перепланирования.
6. **Верификация и слияние.** Оркестратор оценивает результаты (`verifyAndMerge`): годится → финальный ответ; нет → перепланирование (до 3 циклов). Финальная проверка общая: файлы существуют, проверка типов проходит, тесты зелёные — по артефактам всех подзадач.
7. **Результаты — по ссылке.** Если включен режим артефактов, полные ответы субагентов сохраняются в файлы, а плану остаются короткие резюме (см. раздел 11).

Отказоустойчивость всего механизма

Если MoE выключен в конфиге или споткнулся (модель выдала нечитаемый план, оркестратор недоступен) — агент **прозрачно падает обратно** в обычный одиночный режим работы. Задача всё равно будет решена, просто одной моделью без распараллеливания.

8. Как агент не теряет суть проекта

Главная проблема длинных сессий на маленькой модели: контекст заполняется, история сжимается, и агент «забывает», что делал. MMA решает это комбинацией механизмов — **ничего важного не живёт только в контексте**.

Якоря, которые переживают каждое сжатие

При компакции старые сообщения сворачиваются в структурное резюме, но самое критичное дублируется отдельными якорями:

- `[Task: …]` — исходная задача пользователя. Менеджер контекста запоминает *первую настоящую* инструкцию хода (игнорируя служебные обёртки вроде `<system-summary>`) и всегда заново вставляет её в сжатое резюме. Вставленная ошибка или лог ошибку задачей не подменяют. Каждый новый запрос пользователя становится новой сохраняемой задачей (`resetUserTurn()`), чтобы задача прошлого хода не протухала.
- `[Plan: …]` — состояние плана. План вообще не живёт в истории диалога — он лежит на диске (`.mma/plans/active.json`) и после каждого сжатия заново впрыскивается в промпт из первоисточника. Поэтому сколько бы раз ни сжимался контекст, чеклист `[x]/[ ]` остаётся актуальным.
- `[Files: …]` / `[Deleted: …]` / `[Read: …]` — экстрактор фактов из результатов инструментов выписывает созданные/изменённые файлы, удалённые и прочитанные (до 12 последних чтений), так что после сжатия агент знает, что уже видел и что сделал, и не перечитывает одно и то же по кругу.
- `[Errors: …]` — повторявшиеся ошибки тоже попадают в резюме.

Резюме не вкладывается само в себя (сжатые сводки исключаются из экстракции фактов), поэтому якоря не размножаются и не искажаются от сжатия к сжатию.

Почему 10–30 сжатий подряд ничего не ломают

1. **Единственный источник истины — диск, а не контекст.** План, карта проекта, память проекта, артефакты субагентов — всё это файлы на диске. Контекст можно сжать хоть сто раз — при следующем ходе эти данные восстанавливаются из первоисточника.
2. **Защита от «спирали смерти» сжатий.** Каждая компакция снижает оценку качества контекста; раньше после ~50 принудительных сжатий качество падало ниже порога триггера, и сжатия шли бесконечно. Теперь штраф за сжатие ограничен снизу — качество никогда не заклинивает, плюс есть кулдаун между принудительными сжатиями.
3. **Сжатие — не потеря, а переупаковка.** В резюме осознанно сохраняются: задача, список файлов, ошибки, «что пробовали и не сработало», ключевые решения. Теряется только болтовня. Свежие 6 обменов всегда остаются целиком — текущий момент не размывается.
4. **Долговременная память вне сессии.** Повторяющиеся системные сбои (один инструмент упал 3+ раза) автоматически записываются в `errors.md`; решения и соглашения — в `decisions.md` / `conventions.md`. Это markdown-файлы в домашней папке, они живут годами и подмешиваются в промпт новых сессий независимо от того, что было сжато.

Итог: сжатие меняет *форму* памяти (полная история → компактное резюме), но не *факт* знания — потому что всё существенное продублировано якорями, диском или постоянной памятью.

---

9. Память: три горизонта

Память агента многослойная: чем важнее информация и дольше она нужна, тем надёжнее хранится.

Горизонт 1 — рабочий контекст (текущий ход)

Само окно контекста модели. Живёт недолго и периодически сжимается, но критичные факты продублированы якорями `[Task:]`, `[Plan:]`, `[Files:]`, `[Errors:]` (подробно — раздел 8).

Горизонт 2 — сессия на диске (часы и дни)

Каждый диалог сохраняется в `~/.mma/sessions/<id>/`:

- **`meta.json`** — паспорт сессии: имя, модель, счётчики сообщений, временные метки.
- **`history.jsonl`** — переписка (вопросы, ответы, сводки инструментов).
- **`session.jsonl`** — полный журнал для разбора полётов: каждый вызов инструмента с аргументами, сырым результатом и длительностью; события `context` / `compaction` (как заполнялся и сжимался контекст), `llm_usage` (токены и время каждого запроса к модели), аудит-события.

Сессии можно переключать (`/resume` — продолжить позже с тем же контекстом), искать по истории всех сессий инструментом `search_history`. Опционально файлы шифруются (см. раздел «Безопасность»).

Горизонт 3 — постоянная память проекта (месяцы)

Markdown-файлы в домашней папке `~/.mma`, которые переживают любые сессии и подмешиваются в системный промпт каждой новой сессии:

- **`conventions.md`** — соглашения проекта (стиль кода, принятые практики).
- **`decisions.md`** — ключевые архитектурные решения («используем vitest, не jest»).
- **`errors.md`** — выученные ошибки. Пополняется автоматически: если один и тот же инструмент падает 3+ раза за сессию (например, сервер диагностики не стартует из-за битой установки), система записывает правило, чтобы в будущих сессиях агент сразу знал обходной путь.

Плюс явная память по запросу: инструменты `remember` / `recall` — «запомни, что…» / «что ты помнишь о…».

Артефакты и кэши

Ряд рабочих данных тоже живёт на диске внутри проекта: ответы субагентов (`.mma/artifacts/<сессия>/`), планы (`.mma/plans/`), карта проекта (`.mma/index-cache.json`), куки браузера (`.mma/browser/cookies.json`). Всё это — часть той же философии: долговечные знания не держат в контексте.

---

10. Карта проекта (indexer)

Чтобы маленькая модель не гадала («наверное, это Next.js?»), при старте агент строит карту рабочего пространства:

- **Обход проекта**: рекурсивный обходчик определяет языки, считает файлы, вытаскивает ключевые экспорты из кода.
- **Профиль стека**: из манифестов (`package.json` , `pyproject.toml` , `Cargo.toml` , `go.mod` …) читается однострочное резюме `[Stack: …]` — название проекта, главные зависимости, скрипты. Модель сразу видит: Vite+React+TS, а не выдумывает фреймворк.
- **Кэш на диске**: результат индексации пишется в `<проект>/.mma/index-cache.json` , повторные запуски мгновенны.
- **Живая карта**: наблюдатель за файлами замечает изменения и пересобирает карту прямо посреди сессии (карта — динамический блок промпта, а не замороженный снимок со старта).
- **Инструмент** `project_map` для агента: `summary` (языки, топ-каталоги, экспорты), `find` (поиск по путям и именам символов), `refresh` (принудительная переиндексация).

В сумме: агент всегда знает, *что за проект перед ним* и *где что лежит*, даже когда контекст давно сжат — карта восстановима с диска и обновляется сама.

---

11. RLM-приёмы: результаты по ссылке и чанкирование

Идея из статьи про Recursive Language Models — не тащить большие данные через контекст:

- **Артефакты**: полный ответ субагента сохраняется в файл (`.mma/artifacts/<сессия>/`), а родителю уходит короткий путь + резюме. Родитель читает файл целиком только если он действительно нужен. Защита от выхода за пределы каталога включена.
- **chunk_query**: длинный документ (лог, большой файл) режется на куски по границам строк; по каждому куску параллельно запускается простой вопрос-ответ к модели (без полного агентного цикла); результаты опционально синтезируются в итоговый ответ. Ограничение параллелизма настраивается.

---

12. Безопасность (подробно)

Как устроена проверка: конвейер валидации

Каждый вызов инструмента проходит через конвейер:

```
вызов инструмента → плагины → валидаторы → само действие → плагины → результат
```

Порядок валидаторов зависит от типа инструмента: файловые операции проверяются валидатором путей и сканером содержимого, веб-операции — сетевым, команды — валидатором bash. Заблокированный вызов не доходит до файловой системы/сети, а факт блокировки пишется в аудит-журнал.

Важно: безопасность выключена по умолчанию (`security.enabled: false`) — это осознанное решение для обратной совместимости. Включается целиком одним флагом или по частям (у каждого валидатора свой переключатель). Мастер первого запуска применяет ответы пользователя о безопасности к конфигу.

Валидатор команд (bash)

- **Чёрный список команд**: `rm` , `dd` , `chmod` , `wget` , `curl` , `ssh` и т.п. — выполнение отклоняется.
- **Блокировка опасных флагов**: `-rf` , `--force` , `--no-preserve-root` .
- **Лазейки через интерпретаторы закрыты**: нельзя обойти blacklist, спрятав команду внутри `bun -e "..."` или аналогичных флагов интерпретаторов.
- **Белый список (опционально)**: строгий режим, где разрешены только явно перечисленные команды.
- Каждая команда логируется в аудит; на Windows серия повторяющихся запрещённых команд жёстко останавливается, чтобы агент не долбился в закрытую дверь.

Валидатор путей

- **Глобальные запреты**: `.git/` , `node_modules/` , `.env` , `*.key` — чтение и запись запрещены независимо от контекста.
- **Контейнмент**: итоговый (разрешённый) путь обязан лежать внутри рабочей папки агента. Проверка устойчива к хитростям Windows: регистру букв, обоим типам разделителей и подмене через соседний префикс (`docs` vs `docs-other` — второй не пройдёт как «часть» первого).
- **Зоны субагентов**: каждый субагент получает собственный список разрешённых файлов (`allowed_files`) и файлов только-для-чтения (`read_only_files`) — выйти за зону он не может даже случайно.
- Применяется ко всем файловым инструментам. Отдельный нюанс: у MMA есть эвристика «исправления опечаток» в путях — поэтому правило такое, что её результат всегда перепроверяется валидатором, доверять ей самому нельзя.

Валидатор сети

- Чёрный список доменов + опциональный белый список для `web_fetch`, `web_search`, `web_browse`, `download_file` и автоматического поиска по ошибкам.
- **Санитизация URL**: чувствительные query-параметры (токены, ключи в адресной строке) вычищаются до запроса.

Сканер содержимого

Перед записью/правкой файла текст сканируется на опасные паттерны: `eval(`, `child_process.exec(`, `rm -rf` и подобные. Находка помечается, решение — показать предупреждение или заблокировать.

Шифрование сессий

- Файлы `history.jsonl` и `session.jsonl` шифруются алгоритмом **AES-256-GCM** (современный стандарт с проверкой целостности).
- Путь к ключу настраивается в конфиге.
- Обратная совместимость: старые незашифрованные сессии читаются без конвертации.

Аудит-журнал и уведомления

- Все значимые события пишутся в `~/.mma/logs/audit.jsonl`: вызовы инструментов, файловые операции, команды bash, сетевые запросы, блокировки безопасности.
- **Нотификатор**: поверх журнала работает система уведомлений — в файл `audit-notifications.jsonl` или webhook (с автоматическими ретраями при сбоях доставки).
- Фильтрация: по уровню серьёзности (`low` / `medium` / `high` / `critical`) и по типу событий — можно слушать только блокировки, игнорируя рутину.
- Есть статистика: сколько событий, каких типов, за период.

Чистка секретов

Все сообщения логов и аудит-записи автоматически прогоняются через санитайзер: маскируются значения вида `sk-…`, заголовки `Bearer …` и поля с именами `api_key` / `token` / `password` / `secret`. Даже в собственных логах MMA секреты не должны светиться.

Готовые политики

Три пресета одним переключением:

Политика	Для чего	Характер
strict	продакшен, чужие машины	максимум запретов, минимум гибкости
balanced	повседневная работа (по умолчанию)	разумный баланс
permissive	тесты, песочницы	минимум ограничений

Поддерживаются кастомные политики с переопределением отдельных правил.

Прочие ограничители

- Глубина рекурсии субагентов ограничена (по умолчанию 3) — защита от бесконечного порождения субагентами субагентов.
- Лимиты числа файловых операций и частоты запросов (rate limits) — защита от шторма вызовов.

---

13. Диагностика кода (LSP + проверки)

- **LSP** (Language Server Protocol — тот же движок, что подсвечивает ошибки в редакторе): после каждой записи/правки файла агент получает настоящие ошибки компиляции TypeScript/CSS/HTML мгновенно, а не после сборки. Есть отдельная команда «проверь ошибки в проекте». Мёртвый сервер диагностики виден в баннере при старте.
- **lint-on-write**: после каждой записи — быстрая проверка синтаксиса; ошибка приписывается к результату записи, чтобы модель починила сразу же.
- Отсутствующий линтер не спамит ошибками — определяется один раз и пропускается.

---

14. Конвейер анти-галлюцинаций

Перед показом ответа пользователя текст проверяется:

- **Проверка фактов** — упомянутые пути/файлы сверяются с известными (диск, индекс, сжатая память). Понимает dotfiles (`.prettierrc.json`).
- **Проверка консистентности** — не противоречит ли модель решениям, принятым ранее в диалоге.
- **Проверка уверенности** — слишком короткие, повторяющиеся ответы, маркеры неуверенности.

Исходы: пропустить → показать с ⚠️ → перезапросить у модели → заблокировать после 3 попыток и спросить пользователя.

---

15. Модули (подсистемы)

- **Skills** — наборы инструкций по задачам, трёхуровневая progressive disclosure (сначала только названия, детали догружаются по мере нужды).
- **Плагины** — хуки на каждое событие жизненного цикла (до/после размышления, до/после инструмента, стриминг…). Изоляция ошибок: упавший плагин не валит остальных. Версионирование и дедупликация по имени, проверка совместимости с версией MMA. Пример-плагин trace-server транслирует работу агента в браузер в реальном времени.

- **MCP** — подключение внешних инструментов по стандарту Model Context Protocol.
- **Pipelines** — многошаговые сценарии из YAML.
- **Indexer** — см. раздел «Карта проекта».
- **Память** — markdown-заметки между запусками: соглашения проекта, принятые решения, выученные ошибки.
- **Сессии** — несколько независимых диалогов с полным логом на диске; переключение, переименование, поиск по истории всех сессий.
- **Cost tracking** — стоимость сессии в долларах рядом с токенами, разбивка по провайдерам, свои цены для любых моделей.
- **Self-update** — фоновая проверка новой версии на npm и автоустановка; работающий процесс не прерывается.

---

16. Certification

Оффлайн-тест: «работает ли вот эта модель через вот этот бэкенд с MMA?»

- Набор сценариев (создать файл, исправить баг, проверить типы…) прогоняется N раз через реального агента в песочнице.
- Проверка по фактам: файл существует, содержимое корректное, exit code, подстрока в выводе — словам агента не верим.
- Результат в манифесте: галочка ✓ у модели = сертифицирована на этом провайдере этой версией MMA.
- Можно добавлять свои сценарии в YAML.

---

17. REPL (интерактивный режим)

- **Собственный редактор строки** (не стандартный readline, который глючит под Bun): Shift+Enter для многострочного ввода, нормальная вставка из буфера (без дублей экрана), история, навигация по перенесённым строкам.
- Живой чеклист плана печатается при каждом изменении; заголовки инструментов привязаны к шагам плана.
- Горячие клавиши: двойной Esc — прервать текущую операцию, Ctrl+C — выход, Ctrl+V — вставить картинку из буфера.
- Автодеградация в plain-text без TTY/в CI.
- Slash-команды — полный список в разделе 18.

---

18. Команды (полный список)

Одноразовый запуск (без интерактивного режима)

```
mma "<запрос>"              # выполнить запрос и выйти
mma "<запрос>" --json       # тот же прогон, но машиночитаемый JSON-результат
```

Флаги для автоматизации и тестов:

- `--no-agents-md` — не подмешивать AGENTS.md проекта в системный промпт (чистая среда);
- `--exit-on-complete` — завершиться сразу после первого финального ответа;
- `-d <папка>` — рабочая папка агента (песочница).

CLI-подкоманды

Группа	Подкоманды	Что делают
`mma init`	—	первичная настройка проекта
`mma config`	`set`, `show`	изменить ключ конфига / показать текущий конфиг
`mma model`	`list`, `use`	список моделей (с метками сертификации ✓/○/·), выбрать модель
	`certify <имя> [--provider-url] [--tags] [--reps] [--force] [--clean]`	прогнать сертификационный набор сценариев
	`cert-status <имя>`, `cert-list`, `uncertify <имя>`	статус / список / отзыв сертификата
	`context <размер>`	окно контекста модели
`mma provider`	`add <имя> [--url] [--key] [--priority] [--context-window] [--rpm] [--parallel]`	добавить провайдера
	`list`, `use <имя>`	список (с активной меткой `*`) / переключение «на горячую»
	`check [--timeout]`	проверить доступность всех провайдеров
`mma session`	`list`, `show <id>`, `delete <id>`	список / детали / удаление сессий
`mma security`	`status`, `policies`, `set-policy <strict|balanced|permissive>`	текущая конфигурация безопасности / список политик / применение политики
	`enable-encryption`, `disable-encryption`	шифрование файлов сессий вкл/выкл
	`enable-audit`, `disable-audit`, `audit-stats`	аудит-уведомления вкл/выкл / статистика
`mma plugins`	`list [--all]`	внешние плагины (`--all` — вместе со встроенными)

Slash-команды REPL

Команда	Алиасы	Что делает
`/help`		справка по командам
`/run <запрос>`		отправить запрос агенту прямо из REPL
`/image <файл|URL|clipboard>`	`/img`	прикрепить картинку к следующему сообщению
`/config`		сводка: модель, провайдер, бюджеты контекста, лимиты
`/reasoning`		показать/скрыть размышления модели
`/status`		модель, провайдер, активная сессия
`/wizard`	`/setup`	интерактивный мастер настройки (перезаписывает конфиг)
`/sysprompt`		показать собранный системный промпт: токены, блоки, что исключено бюджетом
`/provider [list|use <имя>]`		список провайдеров / переключение без перезапуска
`/model [list|use <имя>]`		модели / смена модели на горячую
`/context <размер>`		изменить окно контекста на лету (минимум 1024)
`/reload`		перечитать конфиг и пересобрать ядро, не выходя из REPL
`/plugins [--all]`		плагины с версиями и источником
`/lsp [status|…]`		состояние серверов диагностики (включённость, отключённые, счётчики сбоев)
`/skill list|loaded|<имя>`		доступные/загруженные скиллы, загрузка по имени
`/sessions`	`/ls`	все сессии (`*` — активная)
`/new <имя>`	`/create`	создать новую сессию
`/resume <id|имя>`	`/switch` , `/use`	переключиться на сессию
`/rename <имя>`		переименовать текущую сессию
`/delete <id>`	`/rm`	удалить сессию
`/clear`		очистить экран
`/exit`	`/quit` , `/q`	выход

---

19. Прочее

- **i18n en/ru** — вообще все пользовательские строки через словарь, оба языка обязаны совпадать по ключам.

- **Автообновление через npm** — см. модули выше.
- **Мультипровайдер** — несколько провайдеров с приоритетами, ценами, лимитами запросов; переключение «на горячую»; автоматический failover.
- **Миграция с v1** — автоопределение старого конфига, бэкап всего `~/.mma`, аккуратный старт с новыми настройками.
- **Диагностика крашей** — глобальный перехват падений с окружением в `crash.jsonl`; отчёт об окружении (ОС, рантайм, инструменты, браузеры) в каждом логе сессии.